

The Invariance Theorem

 Bookmark this page

Problem: Complexity depends on who is measuring it!

At first sight, description complexity seems to be totally subjective. What is simple for you might be complex for me (think of your phone number). More formally, let's consider all binary sequences of size N . There are 2^N of them. Some look fairly simple:

```
0000000000000000000000000000000000000000000000000000000000000000
1111111111111111111111111111111111111111111111111111111111111111
01010101010101010101010101010101010101010101010101010101010101
```

Some look more complex:

0010100101110110101101000101101011011010001011001011

Perspective

1/1 point (ungraded)

We may sort the 2^N different sequences of length N in any order we wish and store them in that order (e.g. into a Python list).

If $N = 52$, how simple can the following sequence become after such an operation?

0010100101110110101101000101101011011010001011001011

- ☒ 1 bit
- ☐ 26 bits
- ☐ 52 bits
- ☐ 53 bits



Answer

Correct:

If that sequence lands at position 1 in the list, one bit is sufficient to represent its rank and so to retrieve it from memory.

Submit

You have used 1 of 1 attempt

i Answers are displayed within the problem

Should we conclude that description complexity is a totally subjective notion? Could it be that some remote mathematical culture would legitimately regard π as complex while other cultures would find it simple?

Is Pi Complex

1/1 point (ungraded)

On March 14th (note the date, 3/14 in US notation), 2019, Emma Haruka Iwao broke the record of the number of decimal digits computed for π . She could determine the first 31 415 926 535 897 digits (over 31 trillions! Note the chosen number) using 96 CPUs during 121 days. In what way can we say that π is *less complex* now than it was in the pre-computer area?

- ☒ in no way.
-
- ☐ those digits are still complex, but less than before as they were nearly impossible to compute.

☐ those digits were complex, but now their complexity is zero.



Answer

Correct:

In no way! There are simple algorithms from which the value of π can be computed, and these algorithms have been known for centuries. The time needed to compute π 's decimals is irrelevant when we speak about Kolmogorov complexity.

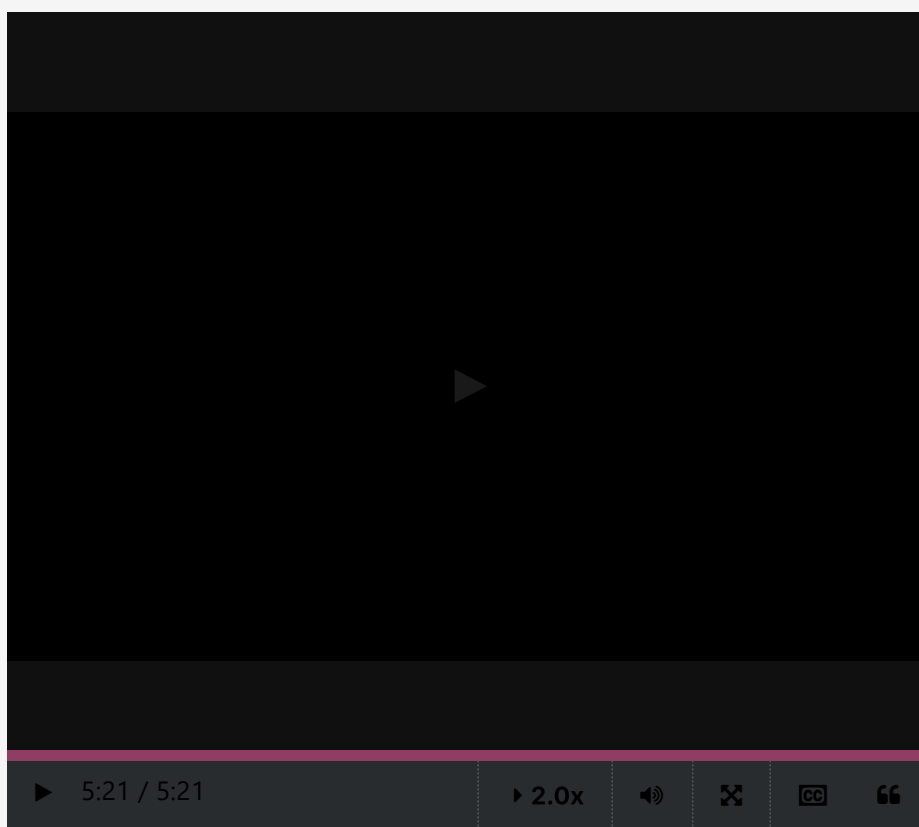
Submit

You have used 1 of 1 attempt

i Answers are displayed within the problem

The **Invariance theorem** puts this idea of "perspective" into perspective.

The invariance theorem



U and $\hat{V}$.

This is why algorithmic information is an objective notion, or, should we say, an almost objective notion.

Conclusion:

Kolmogorov complexity is defined by

considering programs that are run on a

specific universal Turing machine.

But since programs can be translated

if we change the machine

The invariance theorem states that discrepancies in the evaluation of Kolmogorov complexity are bounded.

Invariance theorem

$$\forall s; |C_U(s) - C_V(s)| < c_{U,V}$$

This means that by changing machine U for another universal Turing machine V , the complexity of an object s may be affected at most by a constant $c_{U,V}$, which depends on both machines, but not on s .

Invariance theorem

1/1 point (ungraded)

Does the invariance theorem hold if U and V designate the same universal Turing machine equipped with two different programming languages L_1 and L_2 (such as Java and Python)?

☐ No, because the invariance theorem holds for different machines.

☒ Yes, because we may translate programs from one language into the other.

☐ No, because translation from one language into the other may make some objects much more complex.



Answer

Correct:

The answer is Yes.

If the two languages L_1 and L_2 are "Turing complete" (this is the case for Java and Python), then one can write an interpreter of L_1 using the syntax of L_2 . So any program written in L_1 that produces s can be converted into a program that can run on the machine that has only L_2 . This puts an upper bound $c_{U,V}$ on the difference of complexity. It corresponds to the size of the interpreter.

Submit

You have used 1 of 1 attempt

i Answers are displayed within the problem

The Invariance theorem is good news. It introduces some objectivity in complexity computations, irrespective of the machine or the programming language used. The complexity of π , for instance, will remain finite on any universal Turing machine you consider. π can be computed using a simple series.

$$\pi = 4 * (1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 \dots)$$

In Python, you may approach this series with **the following procedure**, where N controls the number of iterations:

```
def pi(N):
    PiDividedByFour = 0
    for i in range(N):
        PiDividedByFour += (-1)**i / float(2*i+1)
    return 4* PiDividedByFour
```

Computing Pi

0/1 point (ungraded)

Execute the above program for $N = 60000$ and $N = 60001$.

```
$ python Pi.py 60000
```

```
. . .
```

```
$ python Pi.py 60001
```

```
. . .
```

What is the precision we get for π ?

☐ 10^{-3}

☐ 10^{-4}

☒ 10^{-5}



Answer

Incorrect:

```
$ Pi.py 60000
```

```
Approximation for 60000 terms:
```

```
3.1415759869
```

```
$ Pi.py 60001
```

```
Approximation for 60001 terms:
```

```
3.1416093200
```

Due to the alternation of terms in the series, we know that the true value of π lies somewhere between 3.1415759869 and 3.1416093200. So the precision is about 10^{-4} . However, strictly speaking, only the 3 decimals of 3.141 are guaranteed.

Submit

You have used 1 of 1 attempt

✖ Incorrect (0/1 point)

This code provides the N^{th} approximation of π . You may embed this code into an infinite loop, or, if you are *very* patient, you may prefer the recursive version, which you use by calling `pi(0)`:

```
def pi(N): return pi(N+1) + 4*(-1)**N /float(2*N+1)
```

You need to wait for the end of eternity to get the result (provided your computer has infinite memory)! (By the way, our current Python interpreter limits us to 800 recursive calls; you may test yours by adding the exit line: `if N >=800: return 0.`) Does this mean that only Godly Beings may use these algorithmic definitions of π ? In a way yes, but this is not the point. The point is that we do have a **finite definition** (i.e. without "...") of π . This definition provides an upper bound of its complexity $C(\pi)$.

Choosing the machine

In a previous exercise, we changed the code in order to turn a complex sequence into a simple one. We can do this for any object, a book for instance. There is a machine (or a code) V in which the content of a book such as the famous *An introduction to Kolmogorov complexity and its applications* (IKC) has a complexity of only one bit. How can we compress a whole book down to a single bit? This is possible if the book is stored in machine V 's memory. V performs the same computations as a standard universal Turing machine U , but interprets the first bit of any program p in a specific way.

```
# procedure defining V
if p[0] == 0:    print(IKC_content)
else:          return U(p[1:])
```

The machine V defined by the above procedure is a univernal Turing machine, exactly as U . Its particularity is to pay attention to the first bit of any program, just in case its value happens to be 0, in which case it prints IKC. For this strange machine, $C_V(\text{IKC}) = 1$, since 1 bit is sufficient to retrieve the whole book. On another machine U that does not hold IKC in its memory, however, $C_U(\text{IKC})$ is the length of the shortest available identification of the book. If the book can be retrieved from an external resource such as the publisher's Website, complexity might remain limited (e.g. equal to the complexity of the ISBN number). In the worst case in which no external resource is available, the description of the book would contain some compressed version of its content. The consequence is that the $c_{U,V}$ constant may be of the same order of magnitude as the complexities we are interested in.

Notes:

- V holds IKC's content in its memory, whereas a representation of its content must be present in the program fed to U .
- More generally, a machine can use the address of an object in its memory as a code for that object.
- When speaking about compression, people consider a reference machine, as in this [compression contest](#) that can bring you a lot of money.

TinyUrl

1/1 point (ungraded)

Some Web services such as [TinyUrl](#) offer to compress long URLs. You may try to compress two URLs, a short one and a long one. What is your conclusion concerning the compression method used by the service?
(several answers are admissible, choose one.)

- ☐ The system uses a compressor to downsize address length.
- ☐ The system uses a hash table to limit address length.
- ☒ The system assign growing alphanumeric shortcuts to addresses that depends on their order of arrival.

✓

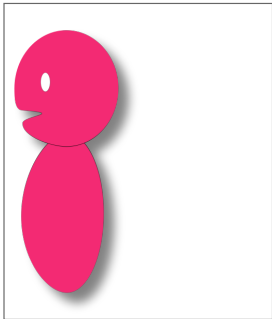
Answer

Correct:
It appears that the system uses a hash table. So very short addresses may end up longer.

Submit

You have used 1 of 1 attempt

✓ Correct (1/1 point)



This makes me think that machine-

< Previous

Next >



edX

- [About](#)
- [Affiliates](#)
- [edX for Business](#)
- [Open edX](#)
- [Careers](#)
- [News](#)

Legal

- [Terms of Service & Honor Code](#)
- [Privacy Policy](#)
- [Accessibility Policy](#)
- [Trademark Policy](#)
- [Sitemap](#)
- [Cookie Policy](#)
- [Your Privacy Choices](#)

Connect

[Idea Hub](#)

[Contact Us](#)

[Help Center](#)

[Security](#)

[Media Kit](#)

